Confluent Kafka Isotope

An example of using Kafka **consumer and producer interceptors** to tag every message with a tracer — an *isotope* — that carries through every hop of a multi-topic pipeline. Apache Flink consumes the tagged records and reports on what the isotopes reveal: end-to-end latency, hop topology, drop/duplication rates, and pipeline coverage.

A **portability requirement** runs through this project: the same isotope mechanism must work against both **Confluent Cloud for Apache Flink** (managed; restricted to Table API SQL + uploaded UDF/PTF JARs) and **Confluent Platform for Apache Flink** (full DataStream + Table API). Three decisions follow from that: tagging happens in the Kafka **producer/consumer interceptors** (the one extension point both runtimes share via the broker), the on-wire **header** format is **JSON** (so Flink SQL can read the scalar fields with `CAST(headers[…] AS STRING)` and no UDF), and the optional stateful reports (`LatencyPercentilesUDAF`, `StuckTracePTF`) ship as a single JAR that registers identically on either runtime.

Message **values** on the demo topics are **SR-framed Protobuf** (`com.life360.kafka.isotope.proto.DemoEvent`) — the standard Confluent value format. The interceptors and reports are agnostic to value format because the isotope rides in headers; the Protobuf choice just gives the integration tests and the demo CLI a typed payload to work with.

How the isotope is carried

- **Header `x-isotope`** (JSON bytes) carries the full hop history, forwarded by every hop:
  - `t` — 16-byte random trace ID, stable for the life of the trace
  - `o` — origin timestamp (ms)
  - `s` — origin service name (set once, never reassigned)
  - `h` — ordered list of hops, each `{s: service, t: topic, m: tsMs}`
  - `x` — `true` if the hop list exceeded `MAX_HOPS = 32` and the oldest hop was evicted
- **Six scalar headers** (UTF-8 strings) carry the most-recent-hop view so Flink SQL can read them via `CAST(headers['x-isotope-…'] AS STRING)` without parsing the JSON array (no UDF required on either CCAF or CP Flink). See flink/README.md for the full header table.

A producer with the isotope interceptor loaded appends one hop on every `send()`. A consume-then-produce service calls `IsotopeContext.adoptFromRecord(record)` between consume and produce so the trace ID and origin survive the hop.

Repo layout

```
app/                                    isotope JVM library + demo CLI +
tests
  src/main/proto/                       DemoEvent.proto (Protobuf message
value)
  src/main/java/com/life360/kafka/isotope/
    Isotope.java                        POJO + JSON codec + Hop +
fromHeaders()
    IsotopeContext.java                 ThreadLocal + adoptFromRecord()
```

```
    IsotopeProducerInterceptor.java     stamps/appends x-isotope + 6 scalar
                                        reporting headers on send()
    IsotopeConsumerInterceptor.java     batch-aware logging; no auto-
propagation
    App.java                            demo CLI — send / hop / sink modes
  src/test/java/.../                    IsotopeCodecTest (no broker needed)
  src/integrationTest/java/.../         live-broker tests; produce/consume
                                        DemoEvent via SR-framed Protobuf
                                        (need Minikube CP + SR port-
forwarded)
ptf/                                    Phase 2 — Flink PTF + UDAF shadow
JAR
  src/main/java/com/life360/kafka/isotope/flink/
    LatencyPercentilesUDAF.java         T-Digest p50/p95/p99 aggregate
    StuckTracePTF.java                  per-trace state + event-time timer
    Percentiles.java, StuckTraceAlert.java  return-type DTOs
  src/test/java/.../                    LatencyPercentilesUDAFTest
flink/sql/                              Flink SQL reports — identical on
                                        CCAF and CP Flink except source DDL
  cp/, cc/, shared/                     per-environment source + shared
reports
k8s/base/                               CFK Kafka/SR/Connect/ksqlDB/C3
manifests
scripts/                                port-forward helpers,
dump_cc_topic.py
Makefile                                cp-up / flink-up / kafka-pf-up /
...
```

Running

1. Unit tests (no broker, instant)

```
./gradlew test                      # both subprojects
# or scoped:
./gradlew :app:test                 # IsotopeCodecTest        — JSON
roundtrip, hop eviction, header size
./gradlew :ptf:test                 # LatencyPercentilesUDAFTest — T-
Digest accumulator semantics
```

2. Demo CLI — see one trace propagate live

The fastest way to watch the isotope mechanic. Requires the cluster to be up and the Kafka + SR forwards running (see step 3 below for the bring-up commands). The CLI has three modes:

Mode	Args	What it does
`send`	`<topic>` `<service>` `<payload>`	Produces one isotope-tagged `DemoEvent` to `<topic>`, then exits. Auto-creates the topic.

Mode	Args	What it does
`hop`	`<in-topic> <out-topic> <service>`	Consumes records from `<in-topic>`, adopts the isotope into thread-local, re-produces the same `DemoEvent` to `<out-topic>` as `<service>` (which appends a new hop). Runs until Ctrl-C.
`sink`	`<topic>`	Subscribes to `<topic>` and pretty-prints the full isotope trail for every arriving record. Runs until Ctrl-C.

A 3-stage chain in four terminals:

```
# Terminal A — terminal sink (will print the full 3-hop trail)
./gradlew :app:run --args="sink iso-final" -q

# Terminal B — middle stage: iso-mid → iso-final as svc-C
./gradlew :app:run --args="hop iso-mid iso-final svc-C" -q

# Terminal C — first stage: iso-start → iso-mid as svc-B
./gradlew :app:run --args="hop iso-start iso-mid svc-B" -q

# Terminal D — kick the chain off (run repeatedly to send more)
./gradlew :app:run --args="send iso-start svc-A 'hello world'" -q
```

Terminal A's output for each record shows the same `trace_id` across all three hops, `origin = svc-A` (never reassigned), and `hops[]` listing `svc-A → svc-B → svc-C` in order with per-hop timestamps. Override endpoints via `-Dkafka.bootstrap=…` / `-Dschema.registry.url=…` if you're not on the default Minikube layout.

3. Integration tests (live Kafka via Minikube)

Bring up the local Confluent Platform stack and port-forward Kafka + SR:

```
make minikube-start                  # one-time
make cp-up                           # CFK Operator +
Kafka/SR/Connect/ksqlDB/C3 (~5 min)
make kafka-pf-up                     # localhost:30092 → Kafka,
localhost:8081 → Schema Registry
```

Then run the suite:

```
./gradlew :app:integrationTest                                  #
all 5 tests
./gradlew :app:integrationTest --tests '*ProducerInterceptorIT'  #
just one
```

Override the endpoints if needed:

```
./gradlew :app:integrationTest \
    -PkafkaBootstrap=localhost:30092 \
    -PschemaRegistryUrl=http://localhost:8081
```

Tear down forwards when done:

```
make kafka-pf-down
```

The integration tests cover:

Test	What it verifies
`BrokerSmokeIT`	AdminClient can create/list/delete a topic via the NodePort port-forward
`ProducerInterceptorIT`	A bare consumer sees the `x-isotope` JSON header + all 6 scalar reporting headers with the expected origin/hop values, and the Protobuf round-trip preserves `DemoEvent.source` / `payload`
`ConsumerInterceptorIT`	`IsotopeContext.adoptFromRecord` extracts isotope into thread-local on tagged records; clears the thread-local for untagged records
`ThreeStageHopPropagationIT`	`svc-A → topic-AB → svc-B → topic-BC → svc-C` produces a stable trace ID, 2-hop trail in send order, and correct scalar headers (origin = `svc-A`, this = `svc-B`, hop count = 2) at the terminal

4. Flink SQL reports

Once interceptor traffic is flowing, the four Phase-1 reports and the two Phase-2 (PTF / UDAF) reports surface latency, topology, hop distribution, coverage, stuck-trace alerts, and p50/p95/p99 percentiles. The reports are identical SQL across Confluent Cloud and Confluent Platform Flink — only the source DDL differs. See **flink/README.md** for the deploy ceremony on each environment.

Recommended path the first time through

1. `./gradlew test` — proves the codec + UDAF logic without any cluster.
2. `make cp-up && make kafka-pf-up && ./gradlew :app:integrationTest` — proves the broker + SR + interceptor + Protobuf path end-to-end.
3. The 3-stage demo CLI walkthrough above — visually shows the trace accumulating hops.
4. `make flink-up` + apply the `.fql` files — reports populate as you drive traffic via the demo CLI.

Status

Piece	Status
Kafka interceptors + JSON codec	Implemented; 6 unit tests passing
Live-broker integration tests	All 5 tests passing against Minikube CP (Kafka 4.x + SR 8.1.0)
Demo CLI (`send` / `hop` / `sink`)	Implemented and exercised
Flink SQL reporting (CC + CP) — Phase 1	4 reports written; awaiting first cluster deploy
PTF stuck-trace + UDAF percentiles — Phase 2	Shadow JAR builds (89 KB); UDAF unit-tested (5 tests); 2 SQL reports written; PTF awaits first live-cluster run for end-to-end verification
Protobuf message values	Implemented via SR; integration tests round-trip `DemoEvent`